Пошаговые игры и паттерн Command

Пошаговые игры и паттерн Command

До сих пор мы говорили о реалтайм-играх: игровой цикл крутится 60 раз в секунду, всё обновляется через deltaTime. Но шахматы, XCOM, Civilization, пошаговые RPG - работают иначе. Игра **ждёт** решения игрока, потом обрабатывает ход, потом ждёт снова.

Как устроен такой цикл? И при чём тут паттерн Command?

Наивный подход

Допустим, у нас RPG: три героя и два гоблина. И ходят они поочереди. Самый простой вариант:

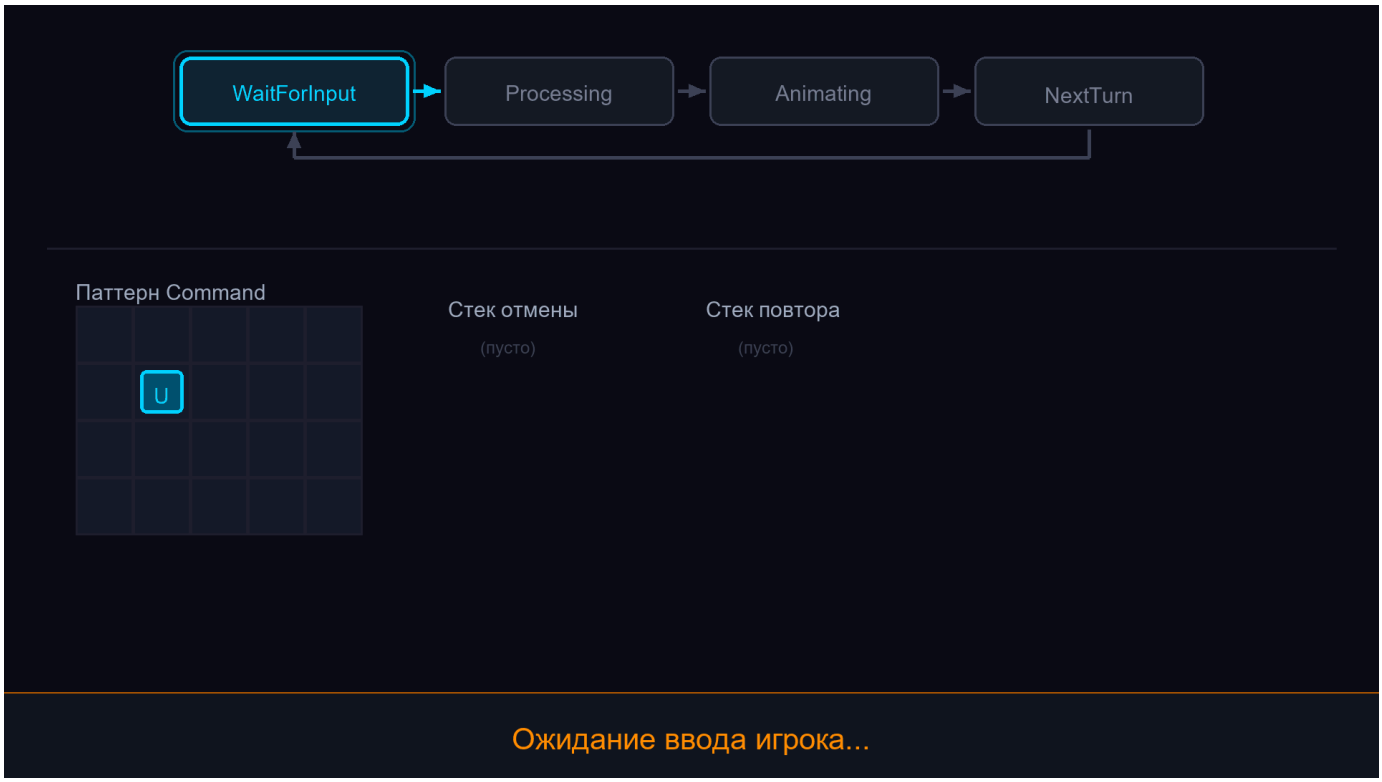
```
while game_running:
    for unit in all_units:
        action = wait_for_input(unit)
        apply(action)
    draw()
```

Цикл блокируется `wait_for_input` - что ж, игровой цикл остановлен ... игра тоже. Пока игрок не кликнет куда-нибудь. Ни анимаций, ни UI, ни курсора. Работает? Формально да. Но ощущение как в (примитивной) консольной программе, а не в игре.

А что, если мы хотим, чтобы во время ожидания ввода юниты покачивались в idle-анимации? Чтобы при наведении подсвечивались клетки? Чтобы после атаки проигрывалась анимация удара, а не просто "HP изменилось"? Нужен игровой цикл, который крутится каждый кадр - как в реалтайме.

Пошаговый цикл

Решение: пошаговость - это не замена реалтайм-циклу, а надстройка поверх него. Цикл по-прежнему тикает 60 раз в секунду, но **игровая логика** продвигается только при переходе между фазами. Получается конечный автомат:



Три фазы:

- **WaitingForInput** - игра ждёт решения. Игрок кликает по клеткам, видит подсветку, думает. ИИ-юнит решает мгновенно. Цикл крутится, анимации idle играют, но логика стоит.
- **Processing** - команда получена, выполняем: переместить юнита, нанести урон, применить эффект.
- **Animating** - ждём, пока анимация покажет результат. Без этой фазы враг телепортируется, здоровье просто меняется - игрок не понимает, что произошло.

Когда анимация закончилась - переключаемся на следующего юнита, возвращаемся в `WaitingForInput`.

Python C#

```python
from enum import Enum


class TurnPhase(Enum):
    WAITING_FOR_INPUT = 1
    PROCESSING = 2
    ANIMATING = 3


class TurnManager:
    def __init__(self, units):
        self.units = units  # all units in turn order
        self.current_index = 0
        self.phase = TurnPhase.WAITING_FOR_INPUT
        self.pending_command = None
```

```
16
17      def current_unit(self):
18          return self.units[self.current_index]
19
20      def submit_command(self, command):
21          """Player (or AI) submits a command."""
22          self.pending_command = command
23          self.phase = TurnPhase.PROCESSING
24
25      def update(self, delta_time):
26          if self.phase == TurnPhase.WAITING_FOR_INPUT:
27              unit = self.current_unit()
28              if unit.is_ai:
29                  # AI decides immediately
30                  command = unit.ai.decide(unit, self)
31                  self.submit_command(command)
32              # else: wait for player input (click, keypress)
33
34          elif self.phase == TurnPhase.PROCESSING:
35              self.pending_command.execute()
36              self.phase = TurnPhase.ANIMATING
37
38          elif self.phase == TurnPhase.ANIMATING:
39              if not animation_playing():
40                  # animation done -> next turn
41                  self.current_index = (self.current_index + 1) % len(self.ur
42                  self.phase = TurnPhase.WAITING_FOR_INPUT
```

Тот же `update`, который вызывается каждый кадр. Но внутри - `switch` по фазам. Грубо говоря, реалтайм-движок работает как обычно, а пошаговость живёт в одном `enum`.

> `animation_playing()` - заглушка для вашей системы анимаций. Реализация зависит от проекта: `return animator.is_playing`, `return anim_timer > 0` или проверка активных твинов. Суть: пока анимация идёт, следующая команда не начинается.

Паттерн Command

В коде выше появляется `command.execute()`. Что это за объект?

Без паттерна Command код выглядит так:

```
1  if clicked_on_tile:
2      unit.x = tile.x
3
```

```
        unit.y = tile.y
```

Прямое изменение. Работает, но: как отменить ход? Как воспроизвести запись партии? Как дать ИИ "подумать" над несколькими вариантами, не меняя реальное состояние?

Command - это действие, упакованное в объект. Вместо "сделай X" мы создаём объект "команда X" и можем его выполнить, отменить, сохранить, передать. Там же мы можем хранить весь необходимый контекст для неё.

Python C#

```python
class Command:
    def execute(self):
        raise NotImplementedError

    def undo(self):
        raise NotImplementedError


class MoveCommand(Command):
    def __init__(self, unit, target_x, target_y):
        self.unit = unit
        self.target_x = target_x
        self.target_y = target_y
        self.prev_x = 0
        self.prev_y = 0

    def execute(self):
        self.prev_x = self.unit.x
        self.prev_y = self.unit.y
        self.unit.x = self.target_x
        self.unit.y = self.target_y

    def undo(self):
        self.unit.x = self.prev_x
        self.unit.y = self.prev_y
```

Каждая команда **запоминает** предыдущее состояние при выполнении. Благодаря этому `undo()` может откатить изменения. `MoveCommand` запомнил, где стоял юнит. По тому же шаблону реализуйте `AttackCommand`: в `execute()` сохраните `prev_hp`, нанесите урон, в `undo()` восстановите. Получается, всё обратимо.

Command - один из паттернов GoF (Gang of Four). В играх он встречается постоянно: ходы в пошаговых играх, системы ввода (привязка клавиш к командам), реплей, сетевая

синхронизация.

Но. У Command есть и минусы. Каждое действие - это отдельный класс, и для сложной игры их может быть десятки. Не все действия легко отменить: если `AttackCommand` зажимает HP к нулю (`hp = max(0, hp - damage)`), простой `hp += damage` уже не вернёт правильное значение - нужно сохранять `prev_hp` . Грубо говоря, чем больше побочных эффектов у действия (звуки, частицы, изменение нескольких объектов), тем сложнее писать корректный `undo()` .

Undo / Redo

Если каждое действие - объект с `execute()` и `undo()` , то отмена хода - это два стека.

Допустим, игрок передвинул воина, потом атаковал гоблина, потом понял, что надо было сначала выпить зелье. Хочет отменить два действия и переиграть.

Реализация: `undo_stack` и `redo_stack` .

- **Execute**: выполняем команду, кладём в `undo_stack` , **очищаем** `redo_stack` . Почему очищаем? Потому что новое действие обрывает цепочку отмен - если вы отменили два хода, а потом сделали что-то новое, "отменённое будущее" больше не существует.
- **Undo**: забираем из `undo_stack` , вызываем `undo()` , кладём в `redo_stack` . Т.е. команда не пропадает - она переезжает в другой стек.
- **Redo**: забираем из `redo_stack` , вызываем `execute()` , кладём обратно в `undo_stack` .

И да. Это та же структура, что используется в текстовых редакторах, графических редакторах, IDE. Ctrl+Z / Ctrl+Y - это ровно этот алгоритм. Одна и та же идея, от шахмат до Photoshop.

Очередь действий

В некоторых пошаговых играх за один ход можно совершить **несколько действий**: переместиться, атаковать, использовать предмет. Или враг на своём ходу делает цепочку: подойти -> ударить -> отступить.

Допустим, враг на своём ходу должен подойти, ударить и отступить. Если выполнить все три команды разом - враг телепортируется, здоровье игрока просто уменьшится, и никто не поймёт, что произошло. Нужна **очередь**, которая выполняет команды по одной, ожидая анимацию между ними.

Получается: враг подошёл - пауза - ударил - пауза - отступил. Игрок видит каждое действие. По сути, очередь превращает мгновенный расчёт в пошаговое кино.

Этот же подход работает для **катсцен**: цепочка команд "камера переместилась -> персонаж сказал реплику -> дверь открылась".

Очерёдность ходов: инициатива

Кто ходит первым? Простой вариант - по очереди: игрок, враг 1, враг 2. Но в RPG обычно порядок определяется характеристикой - **инициативой** (скорость, ловкость, сила, что-нибудь еще).

Это задача для **очереди с приоритетами**. В начале раунда кладём всех юнитов с приоритетом, зависящим от `speed`. Каждый вызов `next_unit()` извлекает самого быстрого. Когда очередь пуста - раунд закончен, начинаем новый.

Нюанс: `heapq` в Python и `PriorityQueue` в C# - это **min-heap**, т.е. извлекают элемент с наименьшим приоритетом. А нам нужен самый быстрый первым. Решение: инвертируем знак. Юнит со speed=10 получает приоритет -10, юнит со speed=5 получает -5. Куча извлечёт -10 первым. Чего-то. Корявенько, но работает.

А что, если мы не хотим фиксированный порядок? Можно добавить случайность: `priority = -speed + random(-2, 2)`. Теперь юнит со speed=10 иногда ходит после юнита со speed=9 - как в настольных RPG, где инициативу бросают кубиком.

Гибриды: реалтайм + пошаговость

Хм. Не всё делится на "чисто реалтайм" и "чисто пошаговое". Много игр комбинируют:

- **Реалтайм с паузой** (Baldur's Gate, Pillars of Eternity) - мир движется в реальном времени, но можно поставить паузу/включить пошаговый режим и раздать команды.
- **Action Points** (XCOM, Divinity) - за ход у юнита есть N очков действия. Перемещение стоит 1 очко, атака - 2. Управляется тем же `ActionQueue`, но с бюджетом.
- **Active Time Battle** (Final Fantasy) - полоска "инициативы" заполняется в реалтайме, ход наступает, когда она полная.
- **Реалтайм с командной очередью** (RTS) - игрок ставит юнитам задачи (shift+click), они выполняются последовательно. Та же `ActionQueue`, но в реалтайме.

Паттерн Command работает во всех этих случаях. Разница только в том, **когда** команды создаются и **как** определяется очерёдность.

Реплей и сетевая игра

Раз все действия - объекты, их можно **сохранить** и **воспроизвести**:

Реплей - это не запись видео. Это список команд. Файл весит килобайты, а не гигабайты.

Тот же принцип используется в **сетевой игре**: клиенты отправляют друг другу не "состояние мира", а команды. Каждый клиент выполняет их у себя и получает одинаковый результат. Это называется **lockstep** - именно так работают Age of Empires, Factorio. Шахматы онлайн тоже обмениваются ходами-командами, хотя и через сервер. Подробнее о сетевых архитектурах - в разделе Сетевая архитектура.

Для lockstep критично, чтобы выполнение команд было **детерминированным**: одна и та же команда на разных компьютерах должна давать одинаковый результат. Floating-point арифметика может подвести - `0.1 + 0.2 ≠ 0.3` на разных платформах.

Немного об ИИ

Паттерн Command - это **абстракция действия**. В reinforcement learning агент выбирает из набора действий (action space) - ровно то, что делает ИИ в пошаговой игре: перебирает возможные команды и выбирает лучшую.

- Минимакс работает именно с командами: "если я сделаю ход X, противник ответит Y, тогда я сделаю Z..." Каждый узел дерева - это команда.
- **MCTS** (Monte Carlo Tree Search) - симулирует тысячи случайных партий. Один из подходов - выполнять и отменять команды через `Undo()`, хотя чаще копируют состояние целиком.
- **Planning в ИИ** - агент строит последовательность действий для достижения цели. `ActionQueue` - это буквально план.

Сравнение

	Реалтайм	Пошаговый
Игровой цикл	Обновляй всё каждый кадр	Жди ввод -> обработай -> покажи результат
deltaTime	Критичен для всей логики	Нужен только для анимаций/UI
ИИ	Решает каждый кадр (FSM, BT)	Решает раз за ход (minimax, planning)
Undo	Обычно нет	Естественно через Command
Сетевая игра	Сложно (интерполяция, lag compensation)	Проще (lockstep, обмен командами)
Примеры	Платформеры, шутеры, экшены	Шахматы, XCOM, Civilization, карточные

Итого

Что	Где подробнее
Игровой цикл и deltaTime	Основной цикл
FSM для фаз хода и состояний юнитов	Конечные автоматы
ИИ противника: FSM, BT	ИИ врагов, деревья поведений
Минимакс для пошаговых решений	Минимакс
Событийная модель для оповещений ("ход начался")	Событийная модель

Что	Где подробнее
Анимации между командами	Анимации
Профилирование ИИ-решений	Отладка

Powered by Wiki.js